Project Documentation: Retail Sales Analytics

Platform	**SAP HANA Cloud**
HDI Schema Name	**SAP_HANA_PROJECTX_HDI_DB_1**

Table of Contents:

1. Business Requirements
2. Data Model
3. Tables (SQL DDL)
4. Sample Data (SQL DML)
5. Calculation Views
6. SQL View
7. SQL Queries
8. SQL Functions
9. Project Structure
10. Security
11. Performance Analysis
12. Summary
13. Files Index

1. Business Requirements

1.1 Business Context

The retail chain sells cosmetics, skincare and hair care products across EMEA, APAC and AMER. With sales data spread across multiple stores, the business needs a consistent way to monitor performance and understand differences between locations, products and customer segments.

1.2 Key Business Questions

- Top 10 stores by net sales (current month)
- Sales trend by month (year-over-year comparison)
- Customer segment analysis (average transaction value)
- Product category performance (margin analysis)

2. Data Model

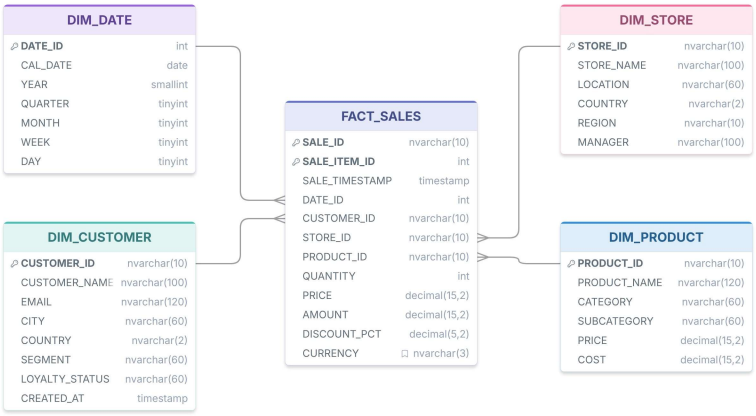
2.1 Data model description

The model uses a central sales transaction data with four dimension tables provide information about customers, products, stores, dates. Sales can be analyzed consistently by product, customer, store, region and time without duplicating descriptive data in the central fact table.

2.2 Data model tables

Table	Type	Purpose	Primary key
DIM_CUSTOMER	Dimension	Customer details, country, segment, loyalty status	CUSTOMER_ID
DIM_PRODUCT	Dimension	Calendar attributes for year, quarter, month, week, day	PRODUCT_ID
DIM_DATE	Dimension	Store name, location, country, region, manager	DATE_ID
DIM_STORE	Dimension	Product name, category, subcategory, price, cost	STORE_ID
FACT_SALES	Fact	Sales line items with quantity, price, amount, discount, currency	SALE_ID, SALE_ITEM_ID

2.3 Data model diagram



3. Tables (SQL DDL)

3.1 Customers

```
COLUMN TABLE DIM_CUSTOMER (
    CUSTOMER_ID VARCHAR(10) PRIMARY KEY,
    CUSTOMER_NAME NVARCHAR(100) NOT NULL,
    EMAIL NVARCHAR(120) NOT NULL,
    CITY NVARCHAR(60) NOT NULL,
    COUNTRY NVARCHAR(2) NOT NULL,
    SEGMENT NVARCHAR(60) NOT NULL,
    LOYALTY_STATUS NVARCHAR(60) NOT NULL,
    CREATED_AT TIMESTAMP DEFAULT CURRENT_TIMESTAMP);
```

3.2 Products

```
COLUMN TABLE DIM_PRODUCT (
    PRODUCT_ID VARCHAR(10) PRIMARY KEY,
    PRODUCT_NAME NVARCHAR(120) NOT NULL,
    CATEGORY NVARCHAR(60) NOT NULL,
    SUBCATEGORY NVARCHAR(60) NOT NULL,
    PRICE DECIMAL(15,2) NOT NULL,
    COST DECIMAL(15,2) NOT NULL);
```

3.3 Date dimension

```
COLUMN TABLE DIM_DATE (
    DATE_ID INT PRIMARY KEY,
    CAL_DATE DATE NOT NULL,
    YEAR SMALLINT NOT NULL,
    QUARTER TINYINT NOT NULL,
    MONTH TINYINT NOT NULL,
    WEEK TINYINT NOT NULL,
    DAY TINYINT NOT NULL);
```

3.4 Stores

```
COLUMN TABLE DIM_STORE (
    STORE_ID VARCHAR(10) PRIMARY KEY,
    STORE_NAME NVARCHAR(100) NOT NULL,
    LOCATION NVARCHAR(60) NOT NULL,
    COUNTRY NVARCHAR(2) NOT NULL,
    REGION NVARCHAR(10) NOT NULL,
    MANAGER NVARCHAR(100) NOT NULL);
```

3.5 Sale line items (main fact)

```
COLUMN TABLE FACT_SALES (
    SALE_ID VARCHAR(10) NOT NULL,
    SALE_ITEM_ID INT NOT NULL,
    SALE_TIMESTAMP TIMESTAMP NOT NULL,
    DATE_ID INT NOT NULL,
    CUSTOMER_ID VARCHAR(10) NOT NULL,
```

```
    STORE_ID VARCHAR(10) NOT NULL,
    PRODUCT_ID VARCHAR(10) NOT NULL,
    QUANTITY INT NOT NULL,
    PRICE DECIMAL(15,2) NOT NULL,
    AMOUNT DECIMAL(15,2) NOT NULL,
    DISCOUNT_PCT DECIMAL(5,2) DEFAULT 0,
    CURRENCY VARCHAR(3) DEFAULT 'EUR',
           PRIMARY KEY (SALE_ID, SALE_ITEM_ID));
```

4. Sample Data (SQL DML)

4.1 Customers [total: 120]

```
INSERT INTO SAP_HANA_PROJECTX_HDI_DB_1.DIM_CUSTOMER VALUES ('C00001', 'Mark Stein', 'mark.stein@example.at', 'Vienna', 'AT',
'New Customer', 'Basic', '2024-01-14 16:09:47.490929139');
INSERT INTO SAP_HANA_PROJECTX_HDI_DB_1.DIM_CUSTOMER VALUES ('C00002', 'Carlos Munoz', 'carlos.munoz@example.es', 'Madrid',
'ES', 'Regular', 'Silver', '2024-01-18 05:33:51.103637086');
INSERT INTO SAP_HANA_PROJECTX_HDI_DB_1.DIM_CUSTOMER VALUES ('C00003', 'Anna Majda', 'anna.majda@example.pl', 'Warsaw', 'PL',
'New Customer', 'Basic', '2024-01-18 07:13:13.875561175');
INSERT INTO SAP_HANA_PROJECTX_HDI_DB_1.DIM_CUSTOMER VALUES ('C00004', 'James Dowson', 'james.dowson@example.com', 'New York',
'US', 'New Customer', 'Basic', '2024-01-22 06:15:56.808402097');
```

4.2 Products [total: 80]

```
INSERT INTO SAP_HANA_PROJECTX_HDI_DB_1.DIM_PRODUCT VALUES ('P0001', 'Hydrating Face Cream 50ml', 'Skincare', 'Face Care',
10.90, 4.50);
INSERT INTO SAP_HANA_PROJECTX_HDI_DB_1.DIM_PRODUCT VALUES ('P0002', 'Anti-Aging Face Cream 50ml', 'Skincare', 'Face Care',
19.90, 8.50);
INSERT INTO SAP_HANA_PROJECTX_HDI_DB_1.DIM_PRODUCT VALUES ('P0003', 'Daily Moisturizer 100ml', 'Skincare', 'Face Care', 8.90,
3.60);
INSERT INTO SAP_HANA_PROJECTX_HDI_DB_1.DIM_PRODUCT VALUES ('P0004', 'Night Repair Cream 50ml', 'Skincare', 'Face Care', 21.90,
9.20);
```

4.3 Date dimension [2024 – 2026]

```
INSERT SAP_HANA_PROJECTX_HDI_DB_1.INTO DIM_DATE
SELECT
    TO_INT(TO_VARCHAR(GENERATED_PERIOD_START, 'YYYYMMDD')) AS DATE_ID,
    GENERATED_PERIOD_START AS "CAL_DATE",
    YEAR(GENERATED_PERIOD_START) AS "YEAR",
    TO_TINYINT(SUBSTRING(QUARTER(GENERATED_PERIOD_START), 7, 1)) AS "QUARTER",
    MONTH(GENERATED_PERIOD_START) AS "MONTH",
    WEEK(GENERATED_PERIOD_START) AS "WEEK",
    DAYOFMONTH(GENERATED_PERIOD_START) AS "DAY"
FROM SERIES_GENERATE_DATE('INTERVAL 1 DAY', '2024-01-01', '2026-10-01');
```

4.4 Stores [total: 120]

```
INSERT INTO SAP_HANA_PROJECTX_HDI_DB_1.DIM_STORE VALUES ('S01', 'Glow & Care Warsaw', 'Warsaw', 'PL', 'EMEA', 'James Taylor');
INSERT INTO SAP_HANA_PROJECTX_HDI_DB_1.DIM_STORE VALUES ('S02', 'Glow & Care Berlin', 'Berlin', 'DE', 'EMEA', 'James Taylor');
INSERT INTO SAP_HANA_PROJECTX_HDI_DB_1.DIM_STORE VALUES ('S03', 'Glow & Care Madrid', 'Madrid', 'ES', 'EMEA', 'James Taylor');
INSERT INTO SAP_HANA_PROJECTX_HDI_DB_1.DIM_STORE VALUES ('S04', 'Glow & Care Paris', 'Paris', 'FR', 'EMEA', 'James Taylor');
```

4.5 Sale line items [total: 600]

```
INSERT INTO SAP_HANA_PROJECTX_HDI_DB_1.FACT_SALES VALUES ('TR000001', 1, '2024-01-14 16:19:24.490929', 20240114, 'C00001',
'S06', 'P0055', 2, 5.90, 11.80, 0.00, 'EUR');
INSERT INTO SAP_HANA_PROJECTX_HDI_DB_1.FACT_SALES VALUES ('TR000001', 2, '2024-01-14 16:19:24.490929', 20240114, 'C00001',
'S06', 'P0021', 2, 9.90, 19.80, 0.00, 'EUR');
INSERT INTO SAP_HANA_PROJECTX_HDI_DB_1.FACT_SALES VALUES ('TR000001', 3, '2024-01-14 16:19:24.490929', 20240114, 'C00001',
'S06', 'P0067', 1, 7.90, 7.90, 0.00, 'EUR');
INSERT INTO SAP_HANA_PROJECTX_HDI_DB_1.FACT_SALES VALUES ('TR000001', 4, '2024-01-14 16:19:24.490929', 20240114, 'C00001',
'S06', 'P0009', 1, 8.50, 8.50, 0.00, 'EUR');
```

5. Calculation Views

5.1 Dimension Calculation Views

Dimension calculation views provide structured access to descriptive data used in sales analysis.

- **CV_DIM_CUSTOMER**: source DIM_CUSTOMER; key CUSTOMER_ID; attributes CUSTOMER_NAME, CITY, COUNTRY, SEGMENT, LOYALTY_STATUS
- **CV_DIM_PRODUCT**: source DIM_PRODUCT; key PRODUCT_ID; attributes PRODUCT_NAME, CATEGORY, SUBCATEGORY, PRICE, COST
- **CV_DIM_DATE**: source DIM_DATE; key DATE_ID; attributes CAL_DATE, YEAR, QUARTER, MONTH, WEEK, DAY
- **CV_DIM_STORE**: source DIM_STORE; key STORE_ID; attributes STORE_NAME, LOCATION, COUNTRY, REGION, MANAGER

5.2 Cube (Star Join) Calculation View: CV_SALES_CUBE

CV_SALES_CUBE is used for analytical queries that evaluate sales measures across business dimensions, such as time and product category. It provides a reusable analytical model with consistent measures and dimensions.

Nodes:

1) Projection: SALES

- Source: FACT_SALES
- Columns: SALE_ID, SALE_ITEM_ID, SALE_DATE, CUSTOMER_ID, STORE_ID, PRODUCT_ID, QUANTITY, PRICE, AMOUNT, DISCOUNT_PCT, CURRENCY
- Calculated measures: NET_SALES, COGS, GROSS_MARGIN

2) Star Join

- Fact: SALES
- Join to CV_DIM_CUSTOMER on CUSTOMER_ID
- Join to CV_DIM_PRODUCT on PRODUCT_ID
- Join to CV_DIM_STORE on STORE_ID
- Join to CV_DIM_DATE on DATE_ID

Calculated measures

These measures support sales performance and profitability analysis

- **SUM(NET_SALES)**: `"QUANTITY"*"PRICE"*(1-"DISCOUNT_PCT")`
  Calculate total sales value after discounts
- **SUM(COGS)**: `"QUANTITY"*"COST"`
  Calculate total cost of products sold
- **SUM(GROSS_MARGIN)**: `"NET_SALES"-"COGS"`
  Calculate total difference between net sales and cogs
- **SUM(QUANTITY)**: Shows total quantity
- **AVG(DISCOUNT_PCT)**: Shows average discount percent

Restricted columns

Together, these measures support comparisons across reporting periods and business regions.

- **SUM(CURRENT_YEAR_SALES)** [measure: NET_SALES]: `"YEAR" = YEAR(CURRENT_DATE)`
- **SUM(LAST_YEAR_ SALES)** [measure: NET_SALES]: `"YEAR" = YEAR(CURRENT_DATE) - 1`
  Restricts values for total net sales to current year and last year sales

- **SUM (AMER_SALES)** [measure: NET_SALES]: `"STORE_REGION" = 'AMER'`
- **SUM (APAC_SALES)** [measure: NET_SALES]: `"STORE_REGION" = 'APAC'`
- **SUM (EMEA_SALES)** [measure: NET_SALES]: `"STORE_REGION" = 'EMEA'`
  Restricts values for total net sales to specific region: AMER, APAC or EMEA

6. SQL View

CV_SALES_ENRICHED is used for queries that require direct SQL control over filtering, transaction level aggregation and ranking. It provides enriched sales data combined with customer, product, date and store attributes.

```
VIEW "CV_SALES_ENRICHED" AS
SELECT
    fs.SALE_ID,
    fs.SALE_ITEM_ID,
    fs.SALE_TIMESTAMP,
    fs.DATE_ID,
    d.CAL_DATE,
    d.YEAR,
    d.MONTH,
    s.STORE_ID,
    s.STORE_NAME,
    s.LOCATION AS STORE_LOCATION,
    s.COUNTRY AS STORE_COUNTRY,
    s.REGION AS STORE_REGION,
    s.MANAGER AS STORE_MANAGER,
    c.CUSTOMER_ID,
    c.CUSTOMER_NAME,
    c.SEGMENT AS CUSTOMER_SEGMENT,
    c.LOYALTY_STATUS AS CUSTOMER_LOYALTY_STATUS,
    p.PRODUCT_ID,
    p.PRODUCT_NAME,
    p.CATEGORY,
    p.SUBCATEGORY,
    fs.QUANTITY,
    fs.PRICE,
    fs.DISCOUNT_PCT,
    (fs.QUANTITY * fs.PRICE * (1 - fs.DISCOUNT_PCT)) AS NET_SALES,
    (fs.QUANTITY * p.COST) AS COGS,
    (fs.QUANTITY * fs.PRICE * (1 - fs.DISCOUNT_PCT)) - (fs.QUANTITY *
```

```
    p.COST) AS GROSS_MARGIN,
    fs.CURRENCY
FROM FACT_SALES fs
LEFT JOIN DIM_CUSTOMER c ON c.CUSTOMER_ID = fs.CUSTOMER_ID
LEFT JOIN DIM_DATE d ON d.DATE_ID = fs.DATE_ID
LEFT JOIN DIM_PRODUCT p ON p.PRODUCT_ID = fs.PRODUCT_ID
LEFT JOIN DIM_STORE s ON s.STORE_ID = fs.STORE_ID;
```

7. SQL Queries

7.1. Top 10 stores by net sales (current month)

Ranks stores by net sales for the current month, helping identify top performing locations and compare sales performance across the retail stores.

```
SELECT
          RANK() OVER( ORDER BY SUM(NET_SALES) DESC ) AS RANK,
          STORE_ID,
          STORE_NAME,
          STORE_LOCATION,
          STORE_COUNTRY,
          STORE_REGION,
          ROUND( SUM(NET_SALES), 2 ) AS NET_SALES
FROM SAP_HANA_PROJECTX_HDI_DB_1.CV_SALES_ENRICHED
WHERE YEAR = YEAR(CURRENT_DATE)
          AND MONTH = MONTH(CURRENT_DATE)
GROUP BY
          STORE_ID,
          STORE_NAME,
          STORE_LOCATION,
          STORE_COUNTRY,
          STORE_REGION
ORDER BY RANK ASC
LIMIT 10;
```

7.2 Sales trend by month (year-over-year comparison)

Compares monthly sales between the current and previous year. Percentage change and trend help identify periods of growth or decline and highlight changes in sales performance.

```
SELECT
          MONTH,
          MONTHNAME(SALE_TIMESTAMP) AS MONTH_NAME,
          CURRENT_YEAR_SALES,
          LAST_YEAR_SALES,
          CURRENT_YEAR_SALES - LAST_YEAR_SALES AS YOY_CHANGE,
          "CALCULATE_YOY_CHANGE_PCT"(CURRENT_YEAR_SALES, LAST_YEAR_SALES) AS YOY_CHANGE_PCT,
          CASE
                    WHEN "CALCULATE_YOY_CHANGE_PCT"(CURRENT_YEAR_SALES, LAST_YEAR_SALES) > 0 THEN 'Increase'
                    WHEN "CALCULATE_YOY_CHANGE_PCT"(CURRENT_YEAR_SALES, LAST_YEAR_SALES) < 0 THEN 'Decrease'
                    ELSE 'No Change'
          END AS TREND
FROM SAP_HANA_PROJECTX_HDI_DB_1.CV_SALES_CUBE;
```

7.3 Customer segment analysis (average transaction value)

Compares customer segments by average transaction value, number of customers and transaction volume. The results help evaluate differences in purchasing behavior across segments.

```
SELECT
          CUSTOMER_SEGMENT,
          COUNT(DISTINCT CUSTOMER_ID) AS TOTAL_CUSTOMERS,
          COUNT(DISTINCT SALE_ID) AS TOTAL_TRANSACTIONS,
          ROUND( SUM(TRANSACTION_VALUE), 2 ) AS TOTAL_NET_SALES,
          ROUND( AVG(TRANSACTION_VALUE), 2 ) AS AVG_TRANSACTION_VALUE
FROM
          (
                    (SELECT
                              CUSTOMER_ID,
                              CUSTOMER_SEGMENT,
                              SALE_ID,
                              SUM(NET_SALES) AS TRANSACTION_VALUE
                    FROM SAP_HANA_PROJECTX_HDI_DB_1.CV_SALES_CUBE
                    GROUP BY
                              CUSTOMER_ID,
                              CUSTOMER_SEGMENT,
                              SALE_ID)
          ) AS TRANSACTIONS
GROUP BY CUSTOMER_SEGMENT
ORDER BY CUSTOMER_SEGMENT DESC;
```

7.4 Product category performance (margin analysis)

Compares product categories and subcategories by net sales, gross margin and margin percentage. This helps identify categories that generate high sales from those that deliver stronger profitability.

```
SELECT
          SUBCATEGORY,
          CATEGORY,
          SUM(GROSS_MARGIN) AS TOTAL_GROSS_MARGIN,
          SUM(NET_SALES) AS TOTAL_NET_SALES,
          "CALCULATE_MARGIN_PCT"( SUM(GROSS_MARGIN), SUM(NET_SALES) ) AS MARGIN_PCT,
          CASE
                    WHEN "CALCULATE_MARGIN_PCT"( SUM(GROSS_MARGIN), SUM(NET_SALES) ) >= 55 THEN 'High'
                    WHEN "CALCULATE_MARGIN_PCT"( SUM(GROSS_MARGIN), SUM(NET_SALES) ) >= 30 THEN 'Medium'
                    ELSE 'Low'
          END AS PERFORMANCE
```

```
FROM SAP_HANA_PROJECTX_HDI_DB_1.CV_SALES_CUBE
GROUP BY
          SUBCATEGORY,
          CATEGORY
ORDER BY MARGIN_PCT DESC;
```

7.5 [Additional query] Daily sales transactions

Provides a transaction level view of sales, allowing individual transactions to be inspected and supporting more detailed analysis of sales activity.

```
SELECT
    SALE_ID,
    SALE_TIMESTAMP,
    NET_SALES,
    CUSTOMER_NAME,
    LAST_DAY(CAL_DATE) AS MONTH_END_DATE,
    TO_VARCHAR(CAL_DATE, 'Day') AS WEEKDAY_NAME
FROM SAP_HANA_PROJECTX_HDI_DB_1.CV_SALES_CUBE
WHERE CAL_DATE = '2026-09-30'
GROUP BY
    SALE_ID,
    SALE_TIMESTAMP,
    NET_SALES,
    CUSTOMER_NAME,
    LAST_DAY(CAL_DATE),
    TO_VARCHAR(CAL_DATE, 'Day')
ORDER BY SALE_TIMESTAMP DESC;
```

7.6 [Additional query] Top 10 products by net sales (current month)

Identifies the products generating the highest net sales in the current month, helping compare product performance and identify the strongest contributors to sales.

```
SELECT
          RANK() OVER( ORDER BY SUM(NET_SALES) DESC ) AS RANK,
          PRODUCT_ID,
          PRODUCT_NAME,
          CATEGORY,
          SUBCATEGORY,
          SUM(QUANTITY) AS TOTAL_QUANTITY,
          ROUND( SUM(NET_SALES), 2 ) AS NET_SALES
FROM SAP_HANA_PROJECTX_HDI_DB_1.CV_SALES_ENRICHED
WHERE YEAR = YEAR(CURRENT_DATE)
          AND MONTH = MONTH(CURRENT_DATE) - 1
GROUP BY
          PRODUCT_ID,
          PRODUCT_NAME,
          CATEGORY,
          SUBCATEGORY
ORDER BY RANK ASC
LIMIT 10;
```

7.7 [Additional query] Sales trend by month and region (current year)

Compares monthly sales across AMER, APAC and EMEA, helping identify regional differences and changes in sales performance over time.

```
SELECT
          MONTH,
          MONTHNAME(SALE_TIMESTAMP) AS MONTH_NAME,
          AMER_SALES,
          APAC_SALES,
          EMEA_SALES,
          SUM(NET_SALES) AS TOTAL_SALES
FROM SAP_HANA_PROJECTX_HDI_DB_1.CV_SALES_CUBE
WHERE YEAR = YEAR(CURRENT_DATE)
GROUP BY
          MONTH,
          MONTHNAME(SALE_TIMESTAMP),
          AMER_SALES,
          APAC_SALES,
          EMEA_SALES
ORDER BY MONTH ASC;
```

8. SQL Functions

SQL GET functions allow users to execute predefined analyses by calling a function instead of rewriting the SQL query each time. It reduces repeated SQL coding and makes recurring sales analyses easier to execute and maintain.

8.1. Top 10 stores by net sales: `"GET_TOP_10_STORES"(YEAR, MONTH)`

```
CREATE FUNCTION "GET_TOP_10_STORES"(
          IN IN_YEAR INTEGER,
          IN IN_MONTH INTEGER )
RETURNS TABLE (
    RANK BIGINT,
    STORE_ID NVARCHAR(50),
    STORE_NAME NVARCHAR(100),
    STORE_LOCATION NVARCHAR(100),
    STORE_COUNTRY NVARCHAR(100),
    STORE_REGION NVARCHAR(50),
    NET_SALES DECIMAL(38,4))
LANGUAGE SQLSCRIPT
AS
```

```
BEGIN
    RETURN
        SELECT
            RANK() OVER (ORDER BY SUM(NET_SALES) DESC) AS RANK,
            STORE_ID,
            STORE_NAME,
            STORE_LOCATION,
            STORE_COUNTRY,
            STORE_REGION,
            ROUND(SUM(NET_SALES), 2) AS NET_SALES
        FROM SAP_HANA_PROJECTX_HDI_DB_1.CV_SALES_ENRICHED
        WHERE YEAR = :IN_YEAR
          AND MONTH = :IN_MONTH
        GROUP BY
            STORE_ID,
            STORE_NAME,
            STORE_LOCATION,
            STORE_COUNTRY,
            STORE_REGION
        ORDER BY RANK ASC
        LIMIT 10;
END
```

8.2 Sales trend by month (year-over-year comparison): "GET_SALES_TREND_YOY"()

```
CREATE FUNCTION "GET_SALES_TREND_YOY"()
RETURNS TABLE (
    MONTH INTEGER,
    MONTH_NAME NVARCHAR(5000),
    CURRENT_YEAR_SALES DECIMAL(25,2),
    LAST_YEAR_SALES DECIMAL(25,2),
    YOY_CHANGE DECIMAL(25,2),
    YOY_CHANGE_PCT DECIMAL(5,2),
    TREND NVARCHAR(20)
)
LANGUAGE SQLSCRIPT
AS
BEGIN
    RETURN
        SELECT
            MONTH,
            MONTHNAME(SALE_TIMESTAMP) AS MONTH_NAME,
            CURRENT_YEAR_SALES,
            LAST_YEAR_SALES,
            CURRENT_YEAR_SALES - LAST_YEAR_SALES AS YOY_CHANGE,
            "CALCULATE_YOY_CHANGE_PCT"(
                CURRENT_YEAR_SALES,
                LAST_YEAR_SALES
            ) AS YOY_CHANGE_PCT,
            CASE
                WHEN "CALCULATE_YOY_CHANGE_PCT"(
                    CURRENT_YEAR_SALES,
                    LAST_YEAR_SALES
                ) > 0 THEN 'Increase'
                WHEN "CALCULATE_YOY_CHANGE_PCT"(
                    CURRENT_YEAR_SALES,
                    LAST_YEAR_SALES
                ) < 0 THEN 'Decrease'
                ELSE 'No Change'
            END AS TREND
        FROM SAP_HANA_PROJECTX_HDI_DB_1.CV_SALES_CUBE;
END;
```

8.3 Customer segment analysis (average transaction value): "GET_CUSTOMER_SEGMENT_ANALYSIS"()

```
CREATE FUNCTION "GET_CUSTOMER_SEGMENT_ANALYSIS"()
RETURNS TABLE (
    CUSTOMER_SEGMENT NVARCHAR(60),
    TOTAL_CUSTOMERS BIGINT,
    TOTAL_TRANSACTIONS BIGINT,
    TOTAL_NET_SALES DECIMAL(25,2),
    AVG_TRANSACTION_VALUE DECIMAL(25,2)
)
LANGUAGE SQLSCRIPT
AS
BEGIN
    RETURN
        SELECT
            CUSTOMER_SEGMENT,
            COUNT(DISTINCT CUSTOMER_ID) AS TOTAL_CUSTOMERS,
            COUNT(DISTINCT SALE_ID) AS TOTAL_TRANSACTIONS,
            ROUND(SUM(TRANSACTION_VALUE), 2) AS TOTAL_NET_SALES,
            ROUND(AVG(TRANSACTION_VALUE), 2) AS AVG_TRANSACTION_VALUE
        FROM (
            SELECT
                CUSTOMER_ID,
                CUSTOMER_SEGMENT,
                SALE_ID,
                SUM(NET_SALES) AS TRANSACTION_VALUE
            FROM SAP_HANA_PROJECTX_HDI_DB_1.CV_SALES_CUBE
            GROUP BY
                CUSTOMER_ID,
                CUSTOMER_SEGMENT,
                SALE_ID
        ) AS TRANSACTIONS
        GROUP BY CUSTOMER_SEGMENT
        ORDER BY CUSTOMER_SEGMENT DESC;
END;
```

8.4 Product category performance (margin analysis): "GET_PRODUCT_CATEGORY_PERFORMANCE"()

```
CREATE FUNCTION "GET_PRODUCT_CATEGORY_PERFORMANCE"()
RETURNS TABLE (
    SUBCATEGORY NVARCHAR(60),
    CATEGORY NVARCHAR(60),
    TOTAL_GROSS_MARGIN DECIMAL(25,2),
```

```
    TOTAL_NET_SALES DECIMAL(25,2),
    MARGIN_PCT DECIMAL(5,2),
    PERFORMANCE NVARCHAR(20)
)
LANGUAGE SQLSCRIPT
AS
BEGIN
    RETURN
        SELECT
            SUBCATEGORY,
            CATEGORY,
            SUM(GROSS_MARGIN) AS TOTAL_GROSS_MARGIN,
            SUM(NET_SALES) AS TOTAL_NET_SALES,
            "CALCULATE_MARGIN_PCT"(
                SUM(GROSS_MARGIN),
                SUM(NET_SALES)
            ) AS MARGIN_PCT,
            CASE
                WHEN "CALCULATE_MARGIN_PCT"(
                    SUM(GROSS_MARGIN),
                    SUM(NET_SALES)
                ) >= 55
                    THEN 'High'
                WHEN "CALCULATE_MARGIN_PCT"(
                    SUM(GROSS_MARGIN),
                    SUM(NET_SALES)
                ) >= 30
                    THEN 'Medium'
                ELSE 'Low'
            END AS PERFORMANCE
        FROM SAP_HANA_PROJECTX_HDI_DB_1.CV_SALES_CUBE
        GROUP BY
            SUBCATEGORY,
            CATEGORY
        ORDER BY MARGIN_PCT DESC;
END
```

8.5 [Additional query] Daily sales transactions: "GET_DAILY_SALES_TRANSACTIONS"(DATE)

```
CREATE FUNCTION "GET_DAILY_SALES_TRANSACTIONS"(
    IN IN_DATE DATE
)
RETURNS TABLE (
    SALE_ID NVARCHAR(10),
    SALE_TIMESTAMP TIMESTAMP,
    NET_SALES DECIMAL(25,2),
    CUSTOMER_NAME NVARCHAR(100),
    MONTH_END_DATE DATE,
    WEEKDAY_NAME NVARCHAR(5000)
)
LANGUAGE SQLSCRIPT
AS
BEGIN
    RETURN
        SELECT
            SALE_ID,
            SALE_TIMESTAMP,
            NET_SALES,
            CUSTOMER_NAME,
            LAST_DAY(CAL_DATE) AS MONTH_END_DATE,
            TO_VARCHAR(CAL_DATE, 'Day') AS WEEKDAY_NAME
        FROM SAP_HANA_PROJECTX_HDI_DB_1.CV_SALES_CUBE
        WHERE CAL_DATE = :IN_DATE
        GROUP BY
            SALE_ID,
            SALE_TIMESTAMP,
            NET_SALES,
            CUSTOMER_NAME,
            LAST_DAY(CAL_DATE),
            TO_VARCHAR(CAL_DATE, 'Day')
        ORDER BY SALE_TIMESTAMP DESC;
END;
```

8.6 [Additional query] Top 10 products by net sales: "GET_TOP_10_PRODUCTS"(YEAR, MONTH)

```
CREATE FUNCTION "GET_TOP_10_PRODUCTS"(
          IN IN_YEAR INTEGER,
          IN IN_MONTH INTEGER )
RETURNS TABLE (
    RANK BIGINT,
    PRODUCT_ID NVARCHAR(50),
    PRODUCT_NAME NVARCHAR(200),
    CATEGORY NVARCHAR(100),
    SUBCATEGORY NVARCHAR(100),
    TOTAL_QUANTITY INT,
    NET_SALES DECIMAL(38,4)
)
LANGUAGE SQLSCRIPT
AS
BEGIN
    RETURN
        SELECT
            RANK() OVER (ORDER BY SUM(NET_SALES) DESC) AS RANK,
            PRODUCT_ID,
            PRODUCT_NAME,
            CATEGORY,
            SUBCATEGORY,
            SUM(QUANTITY) AS TOTAL_QUANTITY,
            ROUND(SUM(NET_SALES), 2) AS NET_SALES
        FROM SAP_HANA_PROJECTX_HDI_DB_1.CV_SALES_ENRICHED
        WHERE YEAR = :IN_YEAR
          AND MONTH = :IN_MONTH
        GROUP BY
            PRODUCT_ID,
            PRODUCT_NAME,
            CATEGORY,
            SUBCATEGORY
```

```
        ORDER BY RANK ASC
        LIMIT 10;
END;
```

8.7 [Additional query] Sales trend by month and region: "GET_MONTHLY_REGIONAL_SALES"(YEAR)

```
CREATE FUNCTION "GET_MONTHLY_REGIONAL_SALES"(
          IN IN_YEAR INTEGER )
RETURNS TABLE (
    MONTH INTEGER,
    MONTH_NAME NVARCHAR(5000),
    AMER_SALES DECIMAL(25,2),
    APAC_SALES DECIMAL(25,2),
    EMEA_SALES DECIMAL(25,2),
    TOTAL_SALES DECIMAL(25,2))
LANGUAGE SQLSCRIPT
AS
BEGIN
    RETURN
        SELECT
            MONTH,
            MONTHNAME(SALE_TIMESTAMP) AS MONTH_NAME,
            AMER_SALES,
            APAC_SALES,
            EMEA_SALES,
            SUM(NET_SALES) AS TOTAL_SALES
        FROM SAP_HANA_PROJECTX_HDI_DB_1.CV_SALES_CUBE
        WHERE YEAR = :IN_YEAR
        GROUP BY
            MONTH,
            MONTHNAME(SALE_TIMESTAMP),
            AMER_SALES,
            APAC_SALES,
            EMEA_SALES
        ORDER BY MONTH ASC;
END;
```

SQL CALCULATE functions define reusable calculation logic, ensuring consistent measures across queries. If a formula needs to be updated, it can be changed in one place instead of modifying each query individually.

8.9. Calculate year-over-year change %: "CALCULATE_YOY_CHANGE_PCT"(CURRENT_YEAR_SALES, LAST_YEAR_SALES)

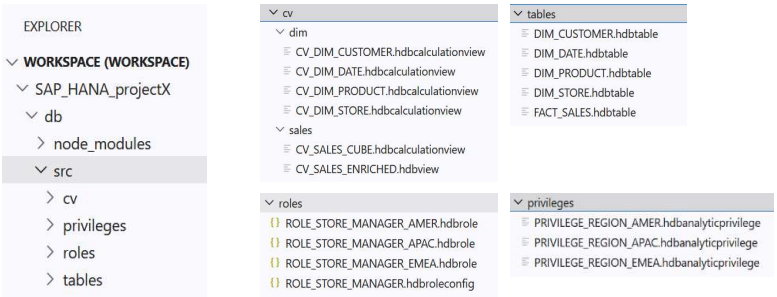
```
CREATE FUNCTION "CALCULATE_YOY_CHANGE_PCT"( IN IN_CURRENT_YEAR_SALES DECIMAL(15, 2), IN IN_LAST_YEAR_SALES DECIMAL(15, 2) )
          RETURNS OUT_YOY_CHANGE_PCT DECIMAL(5, 2)
          LANGUAGE SQLSCRIPT AS
BEGIN
          -- Handle edge cases
          IF :IN_CURRENT_YEAR_SALES = 0
                    OR :IN_LAST_YEAR_SALES = 0 THEN
                    OUT_YOY_CHANGE_PCT := 0;
          ELSE
                    -- Calculate profit margin: (Gross margin / Net sales) * 100
                    OUT_YOY_CHANGE_PCT := ROUND( ( ( :IN_CURRENT_YEAR_SALES - :IN_LAST_YEAR_SALES ) / :IN_LAST_YEAR_SALES )
* 100, 2 );
          END IF;
END
```

8.10. Calculate margin %: "CALCULATE_MARGIN_PCT"(GROSS_MARGIN, NET_SALES)

```
CREATE FUNCTION "CALCULATE_MARGIN_PCT"( IN IN_GROSS_MARGIN DECIMAL(15, 2), IN IN_NET_SALES DECIMAL(15, 2) )
          RETURNS OUT_MARGIN_PCT DECIMAL(5, 2)
          LANGUAGE SQLSCRIPT AS
BEGIN
          -- Handle edge cases
          IF :IN_NET_SALES = 0 THEN
                    OUT_MARGIN_PCT := 0;
          ELSE
                    -- Calculate profit margin: (Gross margin / Net sales) * 100
                    OUT_MARGIN_PCT := ROUND( ( :IN_GROSS_MARGIN / :IN_NET_SALES ) * 100, 2 );
          END IF;
END
```

9. Project Structure

The project organizes tables, views, roles and privileges into separate folders. This structure makes database objects easier to find, supports maintenance, helps keep analytical logic and security configuration organized.



10. Security

The project uses analytic privileges and roles to restrict access to sales data. This allows regional managers to see only their region's data.

10.1 Analytic Privileges

Restrict access to sales data in CV_SALES_CUBE based on STORE_REGION. Each privilege filters the analytical model to the corresponding region.

- **PRIVILEGE_REGION_AMER**: restrict CV_SALES_CUBE.STORE_REGION = 'AMER'
- **PRIVILEGE_REGION_APAC**: restrict CV_SALES_CUBE.STORE_REGION = 'APAC'
- **PRIVILEGE_REGION_EMEA**: restrict CV_SALES_CUBE.STORE_REGION = 'EMEA'

10.2 Roles

Each role grants SELECT access to CV_SALES_CUBE and is associated with the corresponding analytic privilege.

- **ROLE_STORE_MANAGER_AMER**:
  SELECT on CV_SALES_CUBE + assign PRIVILEGE_REGION_AMER
- **ROLE_STORE_MANAGER_APAC**:
  SELECT on CV_SALES_CUBE + assign PRIVILEGE_REGION_APAC
- **ROLE_STORE_MANAGER_EMEA**:
  SELECT on CV_SALES_CUBE + assign PRIVILEGE_REGION_EMEA

11. Performance Analysis

11.1 Performance: CV_SALES_ENRICHED (SQL View)

-- Test query: Top 10 stores by net sales (current month)

Execution Time & Explain Plan

Execution time: 0.884 ms; the query uses joins, aggregation and table scans.

Result × **Messages** × History

```
Statement: SELECT  RANK() OVER( ORDER BY SUM(NET_SALES) DESC ) AS RANK, STORE_ID, STORE_NAME, ...
Client elapsed time:    6.000 ms
Statement prepare time: 0.4430 ms elapsed time, 0.1490 ms CPU time
Statement execute time: 0.8840 ms elapsed time, 0.7560 ms CPU time
Peak memory consumed:   71.51 KB
```

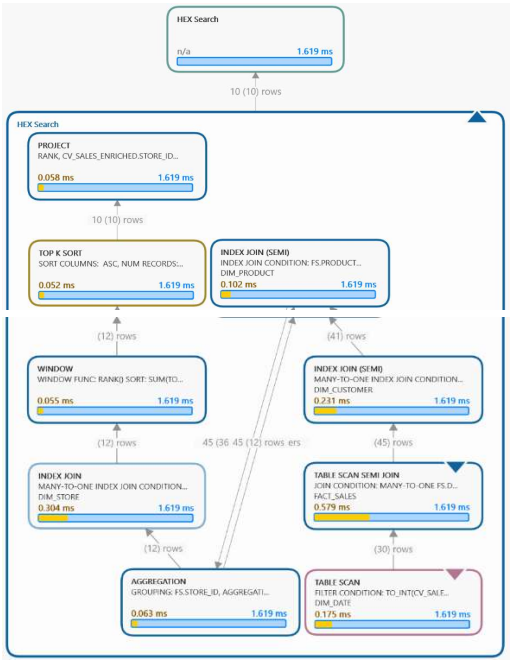
Plan × Messages × History

Rows (9) SQL

	OPERATOR_NAME	OPERATOR_DETAILS	OPERATOR_PROPERTIES	EX...	DA...	SC...	TABLE_NAME	TA...	TA...	OUT...	SUBTREE_COST
1	PROJECT	RANK, CV_SALES_ENRICHED.STORE_I...	PHYSICAL_ENUM_BY: HEX_P...	HEX	NULL	NULL	NULL	NULL	NULL	10	12.00005021580013
2	TOP K SORT	SORT COLUMNS: ASC, NUM RECORDS:...	PHYSICAL_ENUM_BY: HEX_T...	HEX	NULL	NULL	NULL	NULL	NULL	10	12.00005021580013
3	WINDOW	WINDOW FUNC: RANK() SORT: SUM(TO_...	PHYSICAL_ENUM_BY: HEX_...	HEX	NULL	NULL	NULL	NULL	NULL	12.000...	12.000047754861729
4	INDEX JOIN	MANY-TO-ONE INDEX JOIN CONDITION: ...	TRANSLATION TABLE (LEFT): ...	HEX	H00	SAP...	DIM_STORE	COL...	12	12.000...	0.000047754861727626215
5	AGGREGATION	GROUPING: FS.STORE_ID, AGGREGATI...	VALUE_ID GROUPING, PERF...	HEX	NULL	NULL	NULL	NULL	NULL	12.000...	0.000046401386268630666
6	INDEX JOIN (SEMI)	INDEX JOIN CONDITION: FS.PRODUCT_I...	TRANSLATION TABLE (LEFT): ...	HEX	H00	SAP...	DIM_PRODUCT	COL...	80	36.45	0.000042350427913530666
7	INDEX JOIN (SEMI)	MANY-TO-ONE INDEX JOIN CONDITION: ...	TRANSLATION TABLE (LEFT): ...	HEX	H00	SAP...	DIM_CUSTOMER	COL...	120	40.5	0.000037929498873174033
8	TABLE SCAN SEMI JOIN	JOIN CONDITION: MANY-TO-ONE FS.DAT...	TRANSLATION TABLE (RIGHT)...	HEX	H00	SAP...	FACT_SALES	COL...	600	45	0.000033017355495
9	TABLE SCAN	FILTER CONDITION: TO_INT(CV_SALES_...	PHYSICAL_ENUM_BY: HEX_T...	HEX	H00	SAP...	DIM_DATE	COL...	1004	30	0.00003230842884

SQL Analyzer Plan

Total execution time: 2.352 ms; the main CPU operations are table scan semi joins and index joins.





11.2 Performance: CV_SALES_CUBE (Calculation View)

-- Test query: Sales trend by month (year-over-year comparison)

Execution Time & Explain Plan

Execution time: 0.793 ms; the plan includes table scans, joins and aggregation

Result × **Messages** × History

```
Statement: SELECT  MONTH, MONTHNAME(SALE_TIMESTAMP) AS MONTH_NAME, CURRENT_YEAR_SALES, ...
Client elapsed time:    5.000 ms
Statement prepare time: 0.4650 ms elapsed time, 0.1780 ms CPU time
Statement execute time: 0.7930 ms elapsed time, 0.6590 ms CPU time
Peak memory consumed:   65.77 KB
```

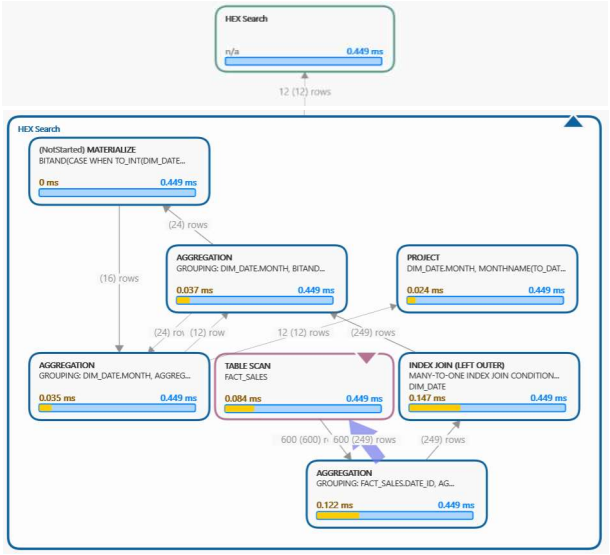
Plan × Messages × History

Rows (7) SQL

	OPERATOR_NAME	OPERATOR_DETAILS	OPERATOR_PROPERTIES	EX...	DA...	SC...	TABLE_N...	TABL...	TA...	OU...	SUBTREE_COST
1	PROJECT	DIM_DATE.MONTH, MONTHNAME(TO...	PHYSICAL_ENUM_BY: HEX_PRO...	HEX	NULL	NULL	NULL	NULL	NULL	12	0.0001696091655200598
2	AGGREGATION	GROUPING: DIM_DATE.MONTH, AGG...	VALUE_ID GROUPING, PERFECT...	HEX	NULL	NULL	NULL	NULL	NULL	12	0.0001696091655200598
3	MATERIALIZE	BITAND(CASE WHEN TO_INT(DIM_DA...	PHYSICAL_ENUM_BY: HEX_MAT...	HEX	NULL	NULL	NULL	NULL	NULL	16	0.0001655860947605979
4	AGGREGATION	GROUPING: DIM_DATE.MONTH, BITA...	ENUM_BY: PREAGGR_BEFORE_...	HEX	NULL	NULL	NULL	NULL	NULL	24	0.0001655860947605979
5	INDEX JOIN (LEFT OUTER)	MANY-TO-ONE INDEX JOIN CONDITIO...	TRANSLATION TABLE (LEFT): FA...	HEX	H00	SAP...	DIM_DATE	COLU...	1004	249	0.0001142020947605979
6	AGGREGATION	GROUPING: FACT_SALES.DATE_ID, A...	VALUE_ID GROUPING, PERFECT...	HEX	NULL	NULL	NULL	NULL	NULL	249	0.0000861174790020000
7	TABLE SCAN		PHYSICAL_ENUM_BY: HEX_TABL...	HEX	H00	SAP...	FACT_SALES	COLU...	600	600	0.0000018

SQL Analyzer Plan

Total execution time: 1.213 ms; the main CPU operations are index joins, aggregation and table scans.





11.3 Performance insights

Both CV_SALES_ENRICHED and CV_SALES_CUBE test scenarios executed successfully with low execution times. All other queries were also tested and showed similar low execution times, indicating consistent performance across different analytical scenarios. No performance issues were observed. These results suggest that the implemented data modeling and optimization practices provide an efficient foundation for sales analysis.

12. Summary

The project brings together data modeling, sales analysis, reusable SQL logic and regional security in SAP HANA Cloud. The implemented solution supports analysis of store and product performance, customer segments, sales trends and profitability. Performance tests showed low execution times across the tested queries, providing a foundation for further development and reporting.

Design and Optimization Decisions

- A star schema separates sales transactions from customer, product, store and date attributes
- Key based joins and appropriate cardinality settings maintain correct relationships between fact and dimension data
- Calculated measures support sales analysis, while restricted columns support comparisons across periods and regions
- Reusable calculation views and the SQL View support different analytical requirements
- CALCULATE functions centralize calculation logic and GET functions simplify recurring sales analyses

- Column pruning, selective column usage and appropriate data types help reduce unnecessary processing
- Analytic privileges and roles restrict access to data by region

Future Improvements: Query performance could be improved through yearly partitioning and union node with pruning. The analytical model could be expanded with additional dimensions, measures and SQL queries to enable deeper analysis of sales trends, customer behavior and product performance. Input variables could allow flexible parameter selection and Node.js application could provide a user friendly interface for reporting and analysis.

13. Files Index

File	Type	Schema	Location
DIM_CUSTOMER.hdbtable	HDI Table	SAP_HANA_PROJECTX_HDI_DB_1	db/src/tables/
DIM_PRODUCT.hdbtable	HDI Table	SAP_HANA_PROJECTX_HDI_DB_1	db/src/tables/
DIM_DATE.hdbtable	HDI Table	SAP_HANA_PROJECTX_HDI_DB_1	db/src/tables/
DIM_STORE.hdbtable	HDI Table	SAP_HANA_PROJECTX_HDI_DB_1	db/src/tables/
FACT_SALES.hdbtable	HDI Table	SAP_HANA_PROJECTX_HDI_DB_1	db/src/tables/
CV_DIM_CUSTOMER.hdbcalculationview	Calculation View	SAP_HANA_PROJECTX_HDI_DB_1	db/src/cv/dim/
CV_DIM_PRODUCT.hdbcalculationview	Calculation View	SAP_HANA_PROJECTX_HDI_DB_1	db/src/cv/dim/
CV_DIM_DATE.hdbcalculationview	Calculation View	SAP_HANA_PROJECTX_HDI_DB_1	db/src/cv/dim/
CV_DIM_STORE.hdbcalculationview	Calculation View	SAP_HANA_PROJECTX_HDI_DB_1	db/src/cv/dim/
CV_SALES_CUBE.hdbcalculationview	Calculation View (Star Join)	SAP_HANA_PROJECTX_HDI_DB_1	db/src/cv/sales/
CV_SALES_ENRICHED.hdbview	SQL View (HDI)	SAP_HANA_PROJECTX_HDI_DB_1	db/src/cv/sales/
PRIVILEGE_REGION_AMER.hdbanalyticprivilege	Analytic Privilege	SAP_HANA_PROJECTX_HDI_DB_1	db/src/privileges/
PRIVILEGE_REGION_APAC.hdbanalyticprivilege	Analytic Privilege	SAP_HANA_PROJECTX_HDI_DB_1	db/src/privileges/
PRIVILEGE_REGION_EMEA.hdbanalyticprivilege	Analytic Privilege	SAP_HANA_PROJECTX_HDI_DB_1	db/src/privileges/
ROLE_STORE_MANAGER_AMER.hdbrole	HDI Role	SAP_HANA_PROJECTX_HDI_DB_1	db/src/roles/
ROLE_STORE_MANAGER_APAC.hdbrole	HDI Role	SAP_HANA_PROJECTX_HDI_DB_1	db/src/roles/
ROLE_STORE_MANAGER_EMEA.hdbrole	HDI Role	SAP_HANA_PROJECTX_HDI_DB_1	db/src/roles/